DOKUZ EYLÜL UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

CME 2201 - Assignment II

GRAPHS IN BIOINFORMATICS

PROJECT REPORT

Prepared by:

2024510014 Emre Can Aras

2024510082 Furkan Türker Yılmaz

Lecturer:

Asst. Prof. Feriştah Dalkılıç

Res. Asst. Orkun Çınar

İzmir/Buca

Date: 24/12/2025

Outline of The Report:

1. Introduction

2. Progress Summary

3. Functions and Solution Strategies

4. References

1. Introduction

In this assignment, we developed a Java program to represent the STRING Protein-Protein Interaction (PPI) Network with a Graph ADT and provide some specified functionalities.

PPI networks provide insights into the complex relationships between proteins and their roles in health and disease. They are crucial in biological research and have various applications across different fields. PPI networks are utilized for understanding biological processes, investigating disease mechanisms, facilitating drug discovery and development, elucidating system biology and network biology, and predicting gene function.

STRING is a database of known and predicted protein-protein interactions. The interactions include direct (physical) and indirect (functional) associations; they stem from computational prediction, from knowledge transfer between organisms, and from interactions aggregated from other (primary) databases. The STRING database currently covers 59,309,604 proteins from 12,535 organisms.[1]

Each interaction in STRING is assigned a confidence score ranging from 0 to 1. This score reflects the reliability of the interaction based on the evidence used to support it.

score >= 0.900	interactions at highest confidence
score >= 0.700	interactions at high confidence or better
score >= 0.400	interactions at medium confidence or better
score < 0.400	low confidence links

Table 1. Confidence Levels of Interactions

An example of the protein networks can be seen in Figure 1.

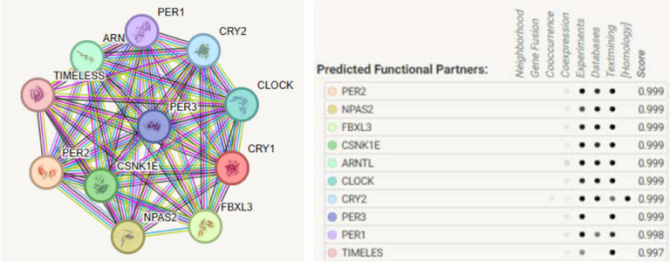

Figure 1. Sample protein network, CRY1. [2]

2. Progress Summary

The project execution timeline spanned from November 28, 2025 (01:30 AM) to December 25, 2025 (11:55 PM). Following the project announcement, our team convened to formulate a strategic development plan.

We began by retrieving the necessary project resources from the Sakai platform. After a thorough analysis of the assignment requirements outlined in the PDF, we selected IntelliJ IDEA as our Integrated Development Environment (IDE) and initialized the project structure. The required datasets that including the protein links and protein info were imported into the project directory.

The implementation phase commenced with the development of file loading operations. We designed the Graph interface and its associated classes.

Subsequently, desired functionalities have been implemented successfully. A User Interface (UI) was integrated to ensure usability. The project was completed on schedule, fully meeting all specified requirements.

3. Functions and Solution Strategies

1. Load Graph:

loadGraph() in UserInterface: Loads the protein links to a graph from the given “9606.protein.links.v12.0.txt” file with using buffer reader and the exception handling has provided with a try catch block. The program asks to user for choose one of the confidence score thresholds and loads the links related to this. Every line in given document has the formula “Protein1/space/Protein2/space/Confidence Score between them” so the program reads every line, splits them from one space, take those proteins as vertices and adds an edge to the Protein1 with confidence score between them as weight.

Vertices stored in a HashMap, which is the IDs are keys and the vertex objects are values. Every vertex keeps its neighbors and edges.

2. Search for protein by ID

The loadProteinsToAnArray() method in the UserInterface class is responsible for loading protein data from the '9606.protein.info.v12.0.txt' file. It instantiates protein objects and

stores them in a hash map, where protein IDs serve as the keys and the objects as the values. During this process, each line of the file is parsed using the tab delimiter ('\t') to populate the respective fields of the protein object. searchByProteinID() in UserInterface takes an ID from the user and checks if the hash map contains it or not, if it contains, prints the protein info.

3. Check if two proteins have an interaction

interactionCheck() in UserInterface: Takes two protein IDs from the user and if vertex hash map contains these IDs, finds the edges between these two proteins and prints if they exist.

hasEdge(String neighbor) in Vertex: It traverses every neighbor of the vertex with Iterator and if it finds the given one, returns true.

getWeight(String destination) in Vertex: It traverses all edge objects associated with the vertex. If a corresponding edge to the destination is identified, the method returns the weight of that edge.

4. Find the most confident path between two proteins

findMostConfidentPath() method inside the UserInterface class takes two protein IDs from the user, controls if they are valid, takes the path from the getMostConfidentPath method as a path object. Then it prints the path.

The getMostConfidentPath(String origin, String destination) method inside the DirectedGraph class, identifies the shortest path with the maximum total weight between two vertices. Starting from the origin, the algorithm traverses the graph by visiting neighbors and utilizing a queue-based approach like exactly the Breath-First traverse. During traversal, it tracks the total weight and cumulative edge count for each visited vertex. Upon reaching the destination, the method evaluates whether the current path offers a lower cost or a higher total weight compared to previously discovered routes. By exhaustively exploring all possible paths, the algorithm determines the shortest path with the maximum total weight. Finally, it returns a Path object, which contains the sequence of vertices stored in a stack and the calculated total weight.

5. Calculate the basic graph metrics (VertexCount, EdgeCount, Average Degree, Diameter, Reciprocity)

Vertex and edge counts are maintained as instance variables within each DirectedGraph object. Whenever a new vertex or edge is added to the graph, these counters increase. The total counts are made accessible through public getter methods, ensuring data encapsulation while allowing other components to retrieve the graph metrics.

The DirectedGraph class also maintains the average degree and reciprocity as instance variables within each object. Upon the addition of a new edge, the average degree is immediately recalculated. Simultaneously, the system utilizes the hasEdge() method to check for the existence of a reciprocal connection; if found, the reciprocal edge count is incremented by two to account for both directions. While these counts are updated during edge insertion, the final reciprocity value is calculated on-demand whenever its respective getter method is called.

Diameter Calculation and Optimization:

The diameter of the graph was calculated using Dijkstra's algorithm. Since the graph can be split into multiple disconnected components based on the selected confidence score thresholds, a standard approach would be insufficient. Therefore, an iterative algorithm was implemented to handle these disjoint subgraphs efficiently.

Two primary methods were designed: findFarthestNodeUsingDijkstra and calculateDiameter.

findFarthestNodeUsingDijkstra: This helper method calculates shortest paths from a given starting vertex to all other reachable vertices. It identifies the farthest vertex and updates the maximum distance found within the component.

calculateDiameter: This method orchestrates the optimization logic. It iterates through the vertices HashMap. For every unvisited vertex, it initiates the Dijkstra process to find the farthest node, then repeats the process starting from that newly found node. This "ping-pong" iteration continues until the calculated distance converges (i.e., the max diameter remains the same as the previous iteration).

To optimize performance and prevent redundant calculations, vertices belonging to a processed disconnected component are marked as visited. The loop skips these visited vertices, ensuring that the expensive Dijkstra operation is only run once per connected component. Finally, the algorithm returns the maximum diameter found among all disconnected components.

6. Breadth-First and Depth-First Traverse by specifying the origin protein

The getBreadthFirstTraversal(String origin) method inside the DirectedGraph implements a breadth-first search algorithm to traverse the graph starting from a specified origin vertex. Initially, all vertices are reset to their unvisited state. The method utilizes two queues: one

to manage the traversal order of vertex labels and another to handle the vertex objects during the exploration process. Starting with the origin, the algorithm systematically visits each neighbor level-by-level, marking them as visited to prevent redundant processing. By using a queue-based approach, the method ensures that all vertices at the current distance are processed before moving to the next level, eventually returning a queue that represents the complete traversal order.

The getDepthFirstTraversal(String origin) method inside the DirectedGraph class implements a depth-first search algorithm to traverse the graph. Similar to the BFS implementation, initially all vertices are reset to their unvisited state. However, instead of a queue, this method utilizes a Stack (LIFO structure) to manage the exploration path. Starting from the origin, the algorithm explores as deep as possible along each branch by pushing unvisited neighbors onto the stack. When a vertex has no remaining unvisited neighbors, the algorithm "backtracks" by popping the vertex from the stack, continuing this process until the stack is empty and the traversal is complete.

7. User Interface and Class Design

In this project, a UserInterface class was designed to serve as a central controller, orchestrating the interaction between the user and the underlying data structures (Graph ADT). This class acts as an abstraction layer, allowing for streamlined execution of graph operations without exposing the internal logic directly to the main execution flow.

The user interaction experience was organized through a menu-driven command-line interface, ensuring clarity in input handling and output presentation. Regarding the class design, a modular architecture was adopted to separate the application logic from the data representation. This separation of concerns significantly enhanced the testability of the system, allowing each component (Vertex, Edge, Graph) to be verified independently.

4. References

[1] STRING, “STRING: functional protein association networks,” *String-db.org*, 2000. https://string-db.org/

[2] “STRING protein-protein interaction networks, CRY1” *String-db.org*, 2015. https://string-db.org/network/9606.ENSP00000008527 (accessed Dec. 23, 2025).